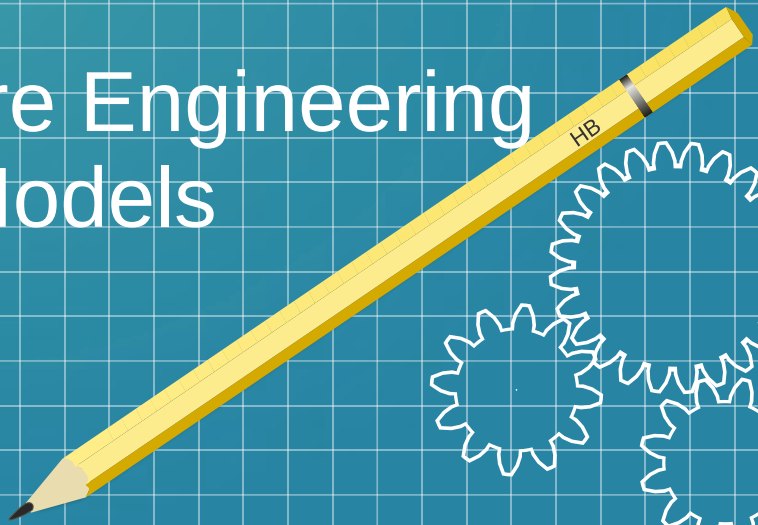
CSC-300: Software Engineering

UNIT-I: Introduction to Software Engineering
Software Life Cycle Models



Software

- Software is
  - **Instructions** that when executed provide desired *features*, *function* and *performance*.
  - **Data structures** that enable the programs to adequately manipulate information.
  - **Descriptive information** in both hard copy and virtual form that describes the operation and use of the programs.

Software characteristics

- Classified into six major components
  - **Functionality**: Degree of performance of the software against its intended purpose.
  - **Reliability**: Ability of the software to provide desired functionality under the given conditions.
  - **Usability**: Extent to which the software can be used with ease.
  - **Efficiency**: Ability to use system resources in the most effective and efficient manner

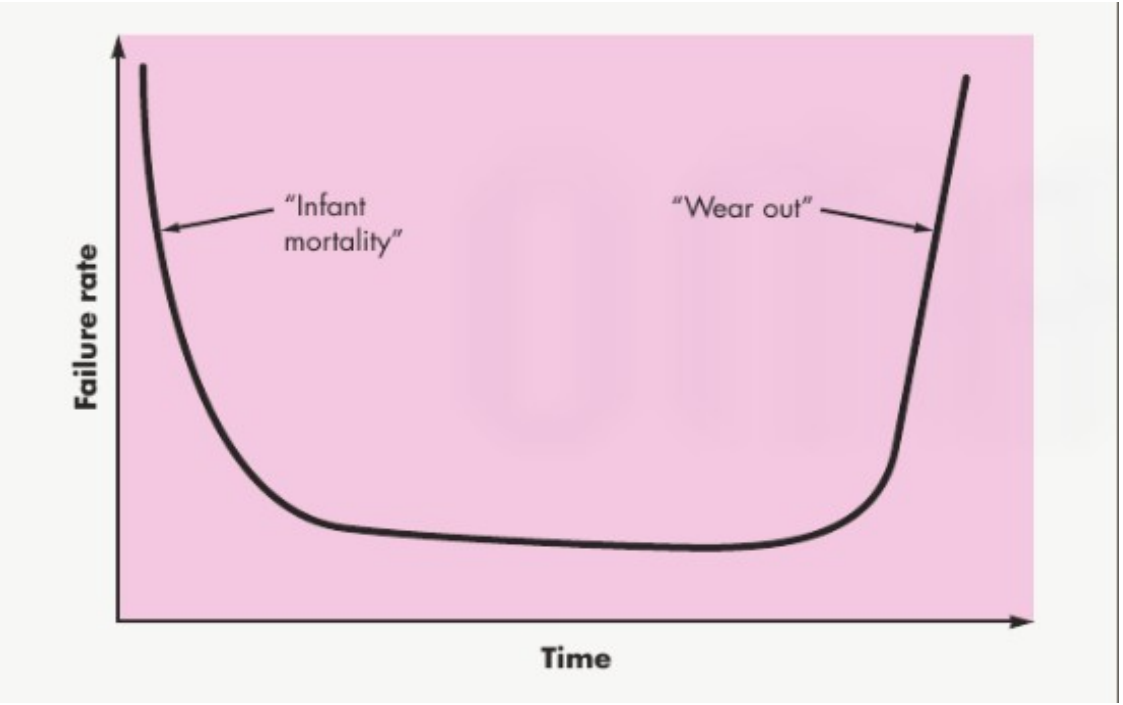
- **Maintainability**: How easily modifications can be made for new features, performance and correct errors.
- **Portability**: How easilty it can be transferred from one platform to another.

- Some more...
  - **Robustness**: Degree to which the software can handle unexpected or bad input without crashing.
  - **Integrity**: Degree to which unauthorized access can be prevented.

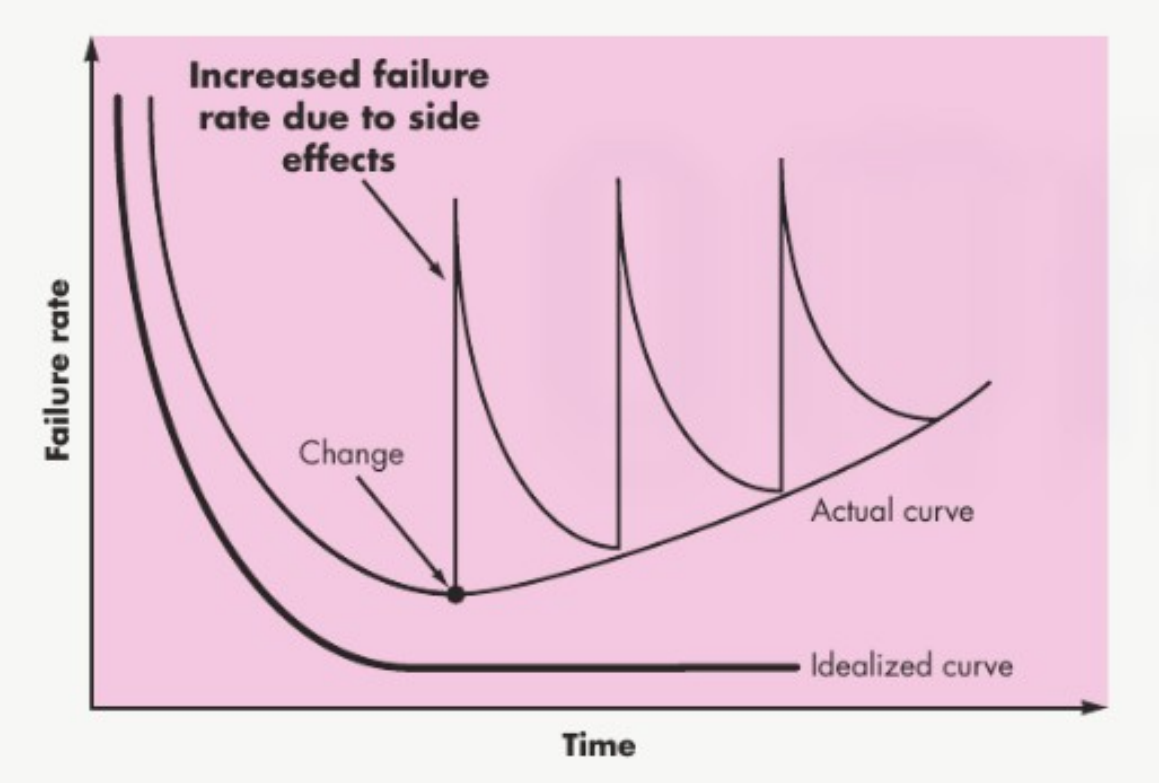
Software vs Hardware characteristics

- *Software is developed or engineered; it is not manufactured in the classical sense*
  - Quality issues during manufacturing phase is non-existent for software
  - Software costs are concentrated in engineering unlike hardware

- *Software doesn’t “wear out”*
  - Failure curve for hardware



- Software doesn’t wear out, but it **deteriorates**.
- Failure curve for software



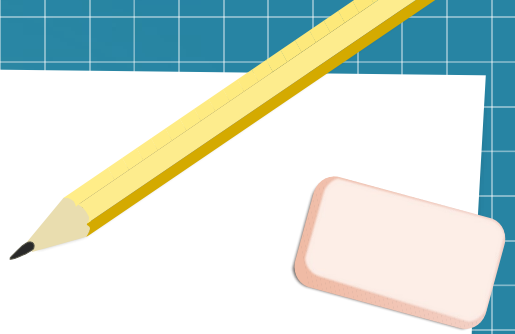
- *Although the industry is moving toward component-based construction, most software continues to be custom built.*
  - *Modern reusable components encapsulate both data and the processing that is applied to the data*

Software Evolution

- ***Software Evolution*** is a term which referes to the process of developing software initially, then timely updating it: add new features, remove obsolete ones etc.

- It includes **change analysis**, **release planning**, **system implementation** and **release**.

- Software evolution is necessary because:
  - **Change in requirement with time**
    - To tackle the change in needs and way of working
  - **Environment change**
    - To tackle the change in surrounding technology or industry
  - **Errors and bugs**
    - To tackle production errors and growing workloads efficiently
  - **Security risks**
    - To tackle cyberattacks and protect sensitive data
  - Etc...



Legacy Software

- *“Legacy software systems ... were developed decades ago and have been continually modified to meet changes in business requirements and computing platforms.”*
  - Dayani-Fard

- Legacy systems sometimes have inextensible designs, convoluted code, poor or nonexistent documentation, test cases and results that were never archived, a poorly managed change history.

Software Engineering

- According to the definition proposed by **Fritz Bauer** at the seminal conference

  "[Software engineering is] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines."

- The **IEEE** has developed a more comprehensive definition when it states:

  "Software Engineering: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (2) The study of approaches as in (1)."

Software Engineering Layers

- Software engineering is a layered technology.
- The bedrock that supports software engineering is a **quality focus**.

FIGURE 1.3

Software engineering layers



Process Layer

- The foundation for software engineering
- The software process forms the basis for management control of software projects
  - work products (models, documents, data, reports, forms, etc.) are produced
  - milestones are established
  - quality is ensured
  - change is properly managed.

Methods Layer

- Software engineering methods provide the technical how-to's for building software.
- Methods encompass a broad array of tasks that include
  - Communication
  - Requirements analysis
  - Design modeling
  - Program construction
  - Testing
  - Support

Tools Layer

- This layer provides automated or semiautomated support for the process and the methods.
- When tools are integrated so that information created by one tool can be used by another, a system for the support of software development, called *computer-aided software engineering*, is established.

The Software Process

- A process is a collection of **activities**, **actions**, and **tasks** that are performed when some work product is to be created.
  - An **activity** strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.
  - An **action** (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural design model).

- A **task** focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

Process framework

- A process framework establishes the foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software projects, regardless of their size or complexity.
- A generic process framework for software engineering encompasses five activities:
  - **Communication**
  - **Planning**
  - **Modeling**
  - **Construction**
  - **Deployment**

Framework Activities

- **Communication**: Before any technical work can commence, it is critically important to communicate and collaborate with the customer (and other stakeholders).

- **Planning**: The map called a software project plan defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.

- **Modeling**: A software engineer creates models to better understand software requirements and the design that will achieve those requirements
- **Construction**: This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.
- **Deployment**: The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

Software Life Cycle

- The life cycle of a software represents the series of identifiable stages through which it evolves during its life time.
- It starts from an initial customer request, to a fully developed software, and finally to a stage where it is no longer useful to any user, and then it is discarded.
- The stage where customer feels the need for software and forms rough ideas about the required features is known as the ***inception stage***. When the user start to use the software, it signals the start of the ***operation*** aka ***maintenance stage***.

Software Development Life Cycle (SDLC) model

- A ***software development life cycle (SDLC) model*** (also called *software life cycle model* and *software development process model*) describes the activities to be carried out for the software to evolve in its life cycle.

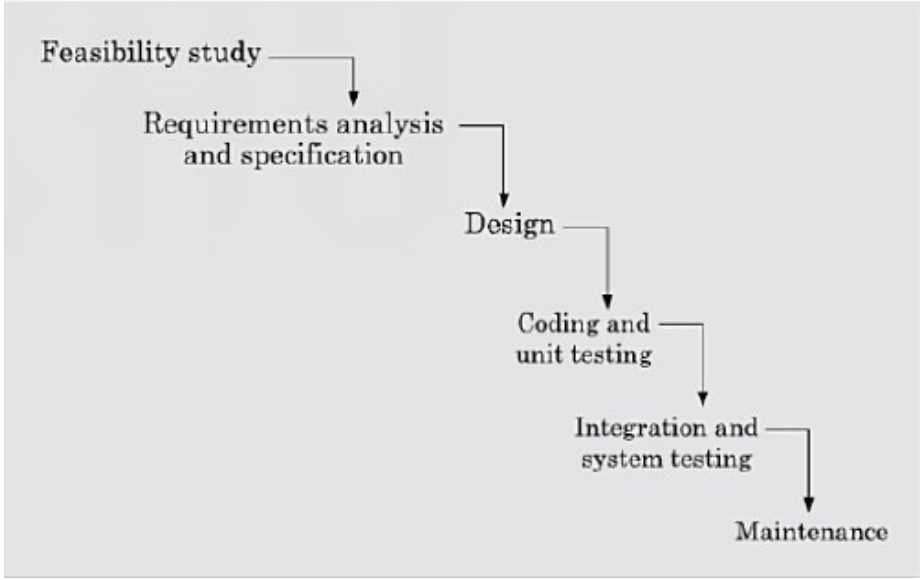
- It depicts graphically the various phases through which software evolves along with textual description on the activities during each phase.

Importance of a life cycle model

- The primary advantage is that it encourages development of software in a systematic and disciplined manner.
- It is especially important to the development of professional software needing team effort.
- If the development team is given the freedom to do whatever, whenever, several problems can arise in interfacing the different parts and manage the overall development.

Classic Waterfall model

- It is the mother of all SDLC models as all can be thought of as extension of the classical waterfall model.
- It divides the life cycle into a set of phases as below:



Phases of the classical waterfall model

- The phases starting from the feasibility study to the integration and system testing phase are known as the ***development phases***.
- After the delivery of software, customers start to use the software signalling the commencement of the ***operation phase***. The last phase is also known as the ***maintenance phase*** of the life cycle.

1. Feasibility study

- The main focus of the feasibility study stage is to determine whether it would be **financially** and **technically feasible** to develop the software.
- The feasibility study involves carrying out several activities such as collection of basic information relating to the software.
- The collected data are analysed to perform the following:
  - **Development of an overall understanding of the problem**: To develop an overall understanding of what the customer requires to be developed.

- **Formulation of the various possible strategies for solving the problem**: Various possible high-level solution schemes to the problem are determined.

- **Evaluation of the different solution strategies**: The different identified solution schemes are analysed to evaluate their benefits and shortcomings. If none of the solutions is feasible due to high cost, resource constraints, or some technical reasons, it would require the project to be abandoned.

2. Requirements analysis and specification

- The aim of the requirements analysis and specification phase is to **understand** the exact requirements of the customer and to **document** them properly.
- This phase consists of two distinct activities, namely ***requirements gathering and analysis***, and ***requirements specification***.

2.1 Requirements gathering and analysis

- It is carried out to collect all relevant information from the customer to have a complete understanding of the requirements.
- For this, the requirements are first **gathered** and then **analysed**.
- The goal is to remove the **incompleteness** and **inconsistencies** in the gathered requirements.

2.2 Requirements specification

- It is the documentation of the identified requirements from the previous stage.
- The produced document is referred to as ***Software Requirements Specification (SRS)*** document
- The SRS document serves as a contract between the development team and the customer.

3. Design

- The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in some programming language.
- In technical terms, during the design phase the *software architecture* is derived from the SRS document.
- Two distinctly different design approaches are popularly being used at present—the **procedural** and **object-oriented design** approaches.

3.1 Procedural design approach

- It is based on data flow-oriented design approach
- It consists of two important activities - **structured analysis** and **structured design**.
  - Structured analysis is where the detailed structure of the problem is examined.
  - Structured design is where the result of structured analysis is transformed into software design.

3.1.1 Structured analysis

- During structured analysis

  1. The functional requirements specified in the SRS document is decomposed into sub-functions.

  2. The data flow between them is analysed and represented diagrammatically in the form of **DFD**s.

3.1.2 Structured design

- It consists of two main activities
  - ***Architectural design*** (aka *high-level design*)
  - ***Detailed design*** (aka *low-level design*)
- High-level design involves decomposing the system into modules, representing the interfaces and the invocation relationships among the modules. It is sometimes referred to as *software architecture*.
- Low-level design, internals of individiual modules such as the data structures and algorithms are designed and documented.

3.2 Object-oriented design approach

- In this approach, objects in the problem and solution domains and the relationships among them are identified.
- This approach is credited to have several benefits such as **lower development time** and **effort** and **better maintainability** of the software.

4. Coding and unit testing

- The purpose of the coding and unit testing phase is to translate a software design into source code and to ensure that individually each function is working correctly.
- Each component of the design is implemented as a **program module**. The end-product of this phase is a set of program modules that have been individually unit tested.
- The specific activities include **designing test cases**, **testing**, **debugging** and **management of test cases**.

5. Integration and system testing

- During this phase, various program modules are integrated in a planned manner.
- Module integration is carried out incrementally over a number of steps. During each step, previous planned modules are added to the partially integrated system and the resultant system is tested.
- After all the modules are integrated successfully and tested, the full working system is obtained.

- Three different types of testing activities in system testing:
  - **Alpha-Testing**: Performed by the development team.
  - **Beta-Testing**: Performed by a friendly set of customers.
  - **Acceptance-Testing**: Performed after final product is delivered, by customers to determine whether to accept or reject the software product.

6. Maintenance

- Maintenance is required in the following three types of situations:
  - **Corrective maintenance**: Maintenance done to correct errors not discovered during development phase.
  - **Perfective maintenance**: Maintenance done to improve the performance or enchance the functionality based on customer’s feedback.
  - **Adaptive maintenance**: Maintenance done to port the software to work in a new environment.

Shortcomings of the classical waterfall model

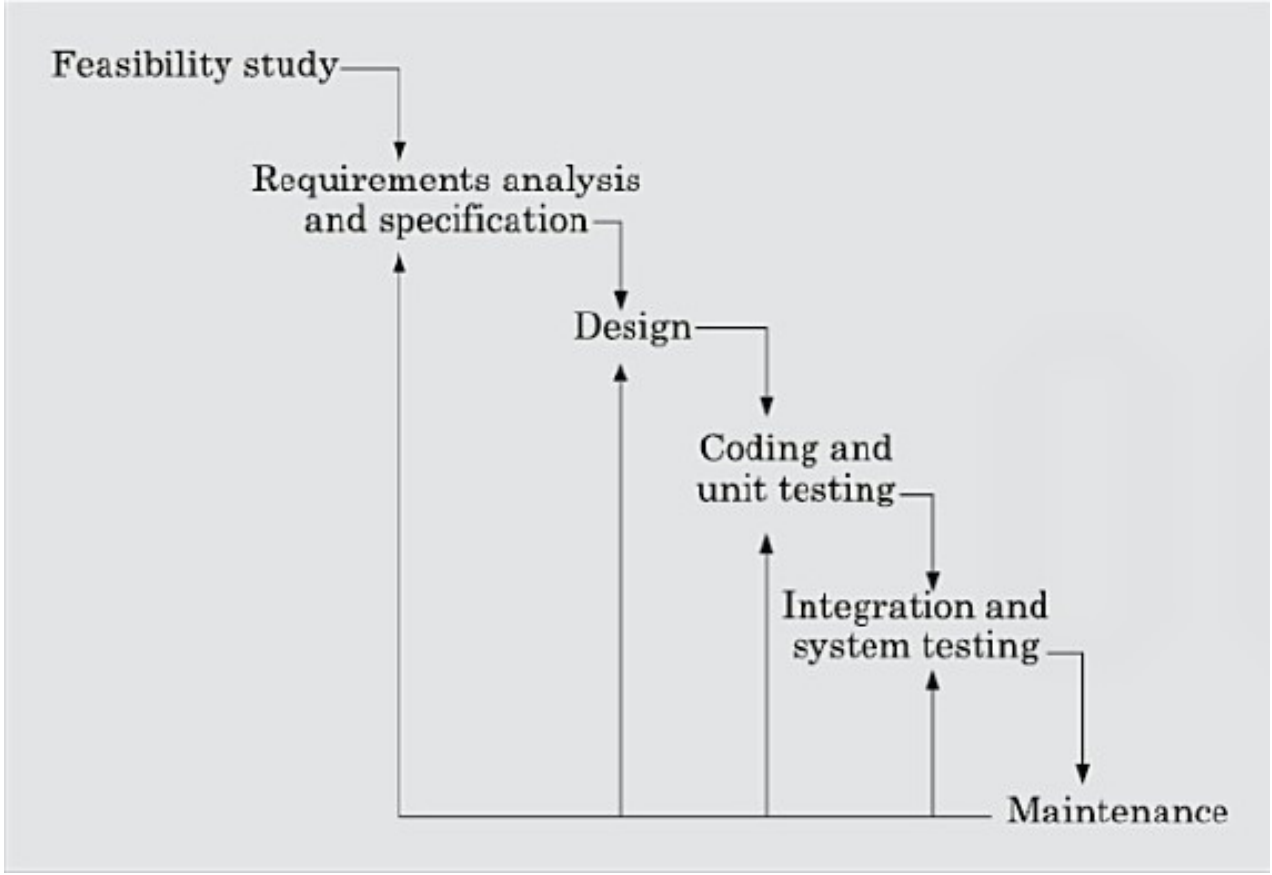
- **No feedback paths**: Once a phase is complete, the activities carried out in it and any artifacts produced in this phase are considered to be final and are closed for any rework.
- **Difficult to accomodate change requests**: This model assumes that all customer requirements can be completely and correctly defined at the beginning of the project.
- **Inefficient error corrections**: This model defers integration of code and testing tasks until it is very late when the problems are harder to resolve

- **No overlapping of phases**: This model recommends that phases be carried out sequentially – while in a practical software development scenario, different phases overlap for cost and efficiency reasons.

Iterative Waterfall Model

- The iterative waterfall model incorporates the necessary changes to the classical waterfall model to make it usable in practical software development projects.

Iterative Waterfall Model - *Diagram*



Iterative Waterfall Model – *Feedback path*

- The feedback paths allow for correcting errors committed by a programmer during some phase, as and when these are detected in a later phase.
- For example, if during the testing phase a design error is identified, then the feedback path allows the design to be reworked and the changes to be reflected in the design documents and all other subsequent documents
- There is no feedback path to the feasibility stage. This is because once a team having accepted to take up a project, does not give up the project easily due to legal and moral reasons..

Iterative Waterfall Model – *Shortcoming*

- **Difficult to accommodate change requests** – Requirements must be fixed before development starts, and detailed plans are built around them.
- **No incremental delivery** – The entire software is built and tested before any delivery happens.
- **No phase overlap** – The model expects each phase to fully finish before the next begins.
- **Expensive error correction** – Testing and validation only happen after development is complete

Iterative Waterfall Model – *Shortcoming (Contd)*

- **Limited customer interaction** – Customers are typically involved only at the very start and end of the project
- **Heavyweight documentation** – The model places heavy emphasis on documentation, which consumes significant time and can reduce team efficiency.
- **Poor support for risk handling and code reuse** – The model isn't well-suited for projects that involve significant risk or that depend on reusing existing code/artifacts

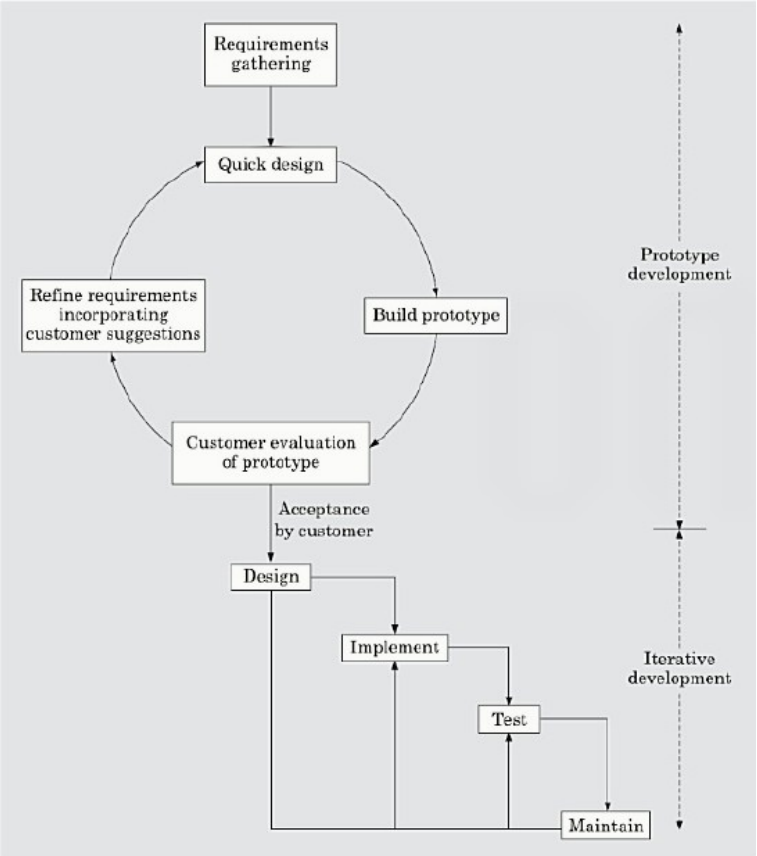
Prototyping Model

- This model suggests building a working prototype of the system, before development of the actual software.
- A prototype can be built very quickly by using several shortcuts. The shortcuts usually involve developing inefficient, inaccurate, or dummy functions.
- Normally the term *rapid prototyping* is used when software tools are used for prototype construction.

Prototyping Model - *Advantages*

- It is advantageous to use the prototyping model for development of the graphical user interface (GUI) part of an application.
- The prototyping model is especially useful when the exact technical solutions are unclear to the development team.
- An important reason for developing a prototype is that it is impossible to “get it right” the first time.

Prototyping Model - *Diagram*



Prototyping Model – *Life Cycle Activities*

- In prototyping model, software is developed through two major activities—**prototype construction** and **iterative waterfall-based software development**.

  1. **Prototype Development**: It starts with initial requirements gathering phase, quick design is carried out and prototype is built. It is then submitted to customers and their feedback is recorded. The requirements are then refined and prototye modified. The cycle continues until the customer approves the prototype.

Prototyping Model – *Life Cycle Activities (Contd)*

2. **Iterative Development**: Once the customer approves the prototype, the actual software is developed using the iterative waterfall approach. The code for the prototype is usually thrown away.

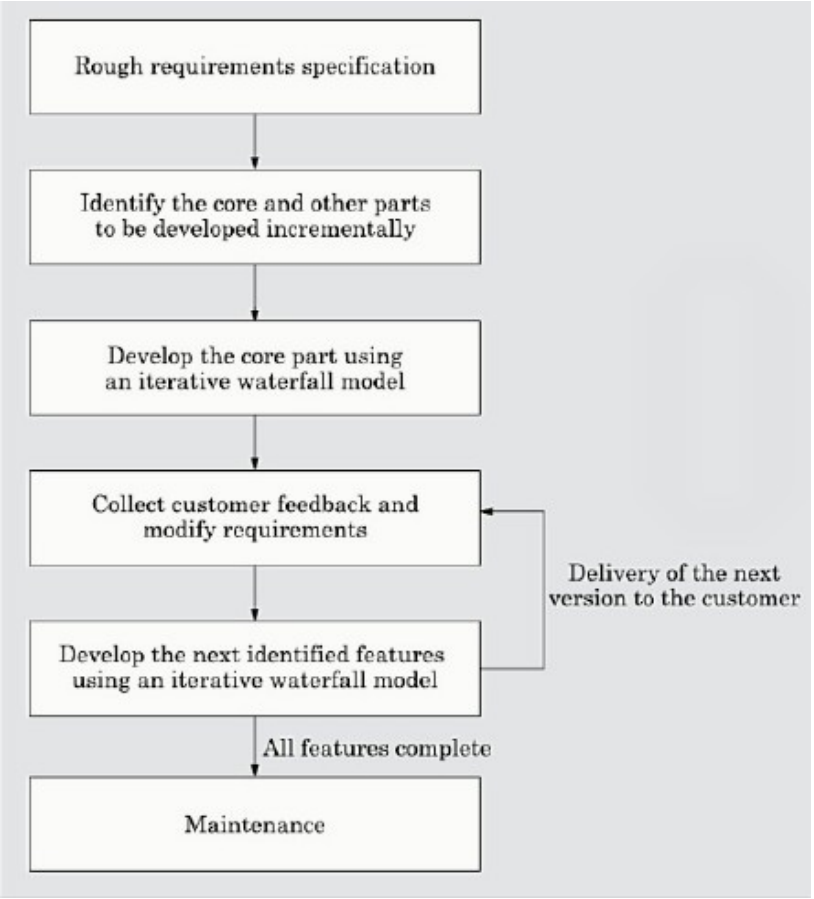
Prototyping Model – *Weaknesses*

- The prototype model can increase the cost of development for projects that are routine development work and do not suffer from any significant risks.
- The prototyping model is effective only for those projects for which the risks can be identified upfront before the development starts.

Evolutionary Model

- In evolutionary model, the software is developed over a number of increments. At each increment, a concept (feature) is implemented and is deployed at the client site.
- The software is successively refined and feature-enriched until the full software is realised.

Evolutionary Model – *Diagram*



Evolutionary Model (Contd)

- In the evolutionary model, the requirements, plan, estimates, and solution evolve over the iterations.
- Such evolution is consistent with the pattern of unpredictable feature discovery and feature changes that take place in new product development.
- The evolutionary software development process is sometimes referred to as *design a little, build a little, test a little, deploy a little model*. i.e., after the requirements have been specified, the design, build, test, and deployment activities are iterated

Evolutionary Model – *Advantages*

- **Effective elicitation of actual customer requirements**: In this model, the user gets a chance to experiment with a partially developed software much before the complete requirements are developed.
- **Easy handling change requests**: In this model, handling change requests is easier as no long term plans are made.

Evolutionary Model – *Disadvantages*

- **Feature division into incremental parts can be non-trivial**: For many development projects, especially for small-sized projects, it is difficult to divide the required features into several parts that can be incrementally implemented and delivered.
- **Ad hoc design**: Since at a time design for only the current increment is done, the design can become ad hoc without specific attention being paid to maintainability and optimality.

Evolutionary Model – *Applicability*

- The evolutionary model is normally useful for very large products, where it is easier to find modules for incremental implementation.
- Often evolutionary model is used when the customer prefers to receive the product in increments
- Another important category of projects for which the evolutionary model is suitable, is projects using object-oriented development since it is easy to partition the software into stand alone units in terms of the classes.

Spiral Model – *Diagram*

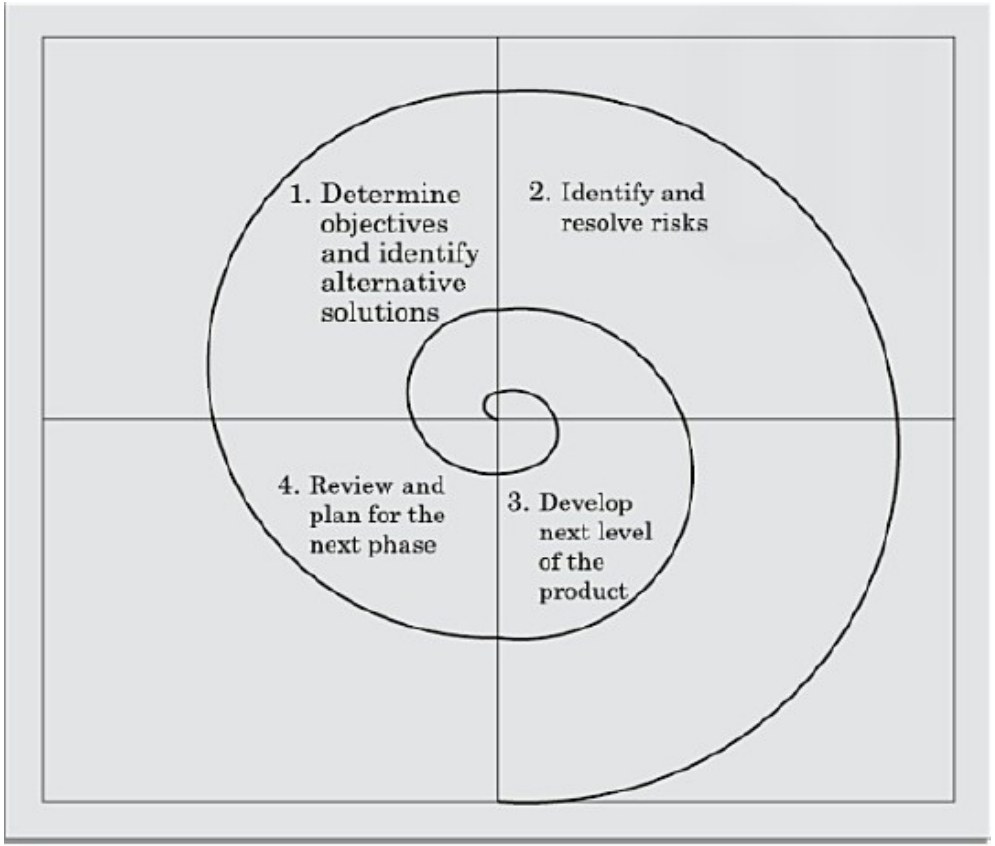


Figure 2.10: Spiral model of software development.

Spiral Model

- Each loop of the spiral is called a ***phase*** of the software process.
- The exact number of phases through which the product is developed can be varied by the project manager depending upon the **project risks**.
- Spiral model requires prototypes to be built at the start of every phase.
- Over each loop, one or more features of the product are elaborated and analysed and the risks at that point of time are identified and are resolved through prototyping. Based on this, the identified features are implemented.

Spiral Model – *Phases*

- Each phase in this model is split into **four sectors** (or ***quadrants***).
  - **Quadrant 1**: The objectives are investigated, elaborated, and analysed. Based on this, the risks involved are identified. Alternative solutions are proposed.
  - **Quadrant 2**: The alternative solutions are evaluated to select the best possible solution. The solutions are evaluated by developing an appropriate prototype.

Spiral Model – *Phases (contd.)*

- **Quadrant 3**: At the end of the third quadrant, the identified features have been implemented and the next version of the software is available.
- **Quadrant 4**: The fourth quadrant concern reviewing the results of the stages traversed so far (i.e. the developed version of the software) with the customer and planning the next iteration of the spiral.

Spiral Model – *To note*

- The radius of the spiral at any point represents the cost incurred in the project so far.
- The angular dimension represents the progress made so far in the current phase.

Spiral Model – *Advantages*

- For projects having many unknown risks that might show up as the development proceeds, the spiral model would be the most appropriate development model to follow.

Spiral Model – *Disadvantages*

- To the developers of a project, the spiral model usually appears as a complex model to follow, since it is risk-driven and is more complicated phase structure.
- It is not very suitable for use in the development of outsourced projects

Agile Model

- The agile software development model was proposed in the mid-1990s to overcome the serious shortcomings of the waterfall model of development.
- The agile model was primarily designed **to help a project to adapt to change requests quickly**.
- A major aim of the agile models is **to facilitate quick project completion**.
- **Agility is achieved by fitting the process to the project**, i.e. removing activities that may not be necessary for a specific project. Also, anything that that wastes time and effort is avoided.

Agile Model – *Development Processes*

- **Agile model is used as an umbrella term to refer to a group of development processes**. These processes share certain common characteristics, but do have certain subtle differences among themselves.
- A few popular agile sdlc models are the following:
  - Crystal
  - Scrum
  - Extreme Programming (XP)
  - Etc...

Agile Model – *Working*

- In the agile model, the requirements are decomposed into many small parts that can be incrementally developed.
- Each iteration is intended to be small and easily manageable and lasting for a couple of weeks only.
- At a time, only one increment is planned, developed, and then deployed at the customer site.

Agile Model – *Time Box*

- The time to complete an iteration is called a ***time box***.
- The implication of the term time box is that the end date for an iteration does not change.
- The development team can, however, decide to reduce the delivered functionality during a time box if necessary.

Agile Model (*contd.*)

- For establishing close contact with the customer during development each agile project usually includes a customer representative in the team.
- At the end of each iteration, stakeholders and the customer representative review the progress made and re-evaluate the requirements.
- Agile model emphasise face-to-face communication over written documents.

Agile Model (*contd.*)

- It is recommended that the development team size be deliberately kept small (5–9 people).
- Agile development projects usually deploy ***pair programming***. Where two programmers work together at one work station. One types in code while the other reviews the code as it is typed in. The two programmers switch their roles every hour or so.

Agile Model – *Disadvantages*

- Lack of adequate documentation may lead to several types of problems, which are as follows:
  - Lack of formal documents leaves scope for confusion and important decisions taken during different phases can be misinterpreted at later points of time by different team members.
  - It becomes difficult to get important project decisions such as design decisions to be reviewed by external experts.
  - When the project completes and the developers disperse, maintenance can become a problem.

Agile Model – *Agile vs Iterative waterfall model*

- The waterfall model is highly structured and systematically steps through its various phases in a planned sequence.
- Progress is generally measured in terms of the number of completed and reviewed artifacts.
- In contrast, agile model if very flexible in the way it only has phases selected as appropriate for the project.
- While using an agile model, progress is measured in terms of the developed and delivered functionalities.

Agile Model – *Agile vs Iterative waterfall model (contd.)*

- If a project being developed using waterfall model is cancelled mid-way, then there is nothing to show beyond several documents.
- With agile model, even if a project is cancelled midway, it still leaves the customer with some worthwhile code, that might possibly have already been put into live operation.

Scrum Model

- In the scrum model, **a project is divided into small parts of work that can be incrementally developed**
- **These parts are then delivered over time boxes that are called *sprints*.**
- **A sprint typically takes only a couple of weeks to complete.**
- **At the end of each sprint, stakeholders and team members meet to assess the progress made**
- The **stakeholders suggest any changes needed to features and any overall improvements that they might feel necessary**

Scrum Model

- In the scrum model, the team members assume three fundamental roles—
  - **Software Owner:** He/she is **responsible for communicating the customers vision of the software to the development team**
  - **Scrum Master:** He/she **acts as a liaison between the software owner and the team**
  - **Team Member**